

Project Report

Distributed E-Commerce System with Database Fragmentation and Replication

Distribuy Project Team

May 2026

1. Introduction

Distribuy is a Distributed E-Commerce System designed as a comprehensive academic project demonstrating advanced distributed database concepts. The system showcases a scalable e-commerce platform built with modern database technologies, implementing key distributed systems principles including:

- **Functional fragmentation** across multiple database services
- **Database replication** using PostgreSQL streaming replication and MongoDB replica sets
- **Distributed transaction handling** via Two-Phase Commit (2PC) simulation
- **Failover simulation** and real-time monitoring capabilities
- An **admin dashboard** for system monitoring and management

The project serves as a practical implementation of Advanced Database Management System (ADBMS) concepts, illustrating how distributed databases can be architected for real-world e-commerce applications.

2. Methodology

The Distribuy system follows a **microservices-based architecture** with functional fragmentation by domain. The methodology encompasses:

2.1. System Architecture Approach

The system is built around four core services, each managing a separate data domain:

1. **User Service** — Handles authentication, user profiles, and audit logging
2. **Order Service** — Manages order lifecycle, order items, and status tracking
3. **Inventory Service** — Controls stock management and inventory transactions
4. **Product Catalog Service** — Provides product information and reviews

2.2. Distributed Transaction Management

The system implements a **Two-Phase Commit (2PC) simulation** to coordinate transactions across the Inventory and Order services:

- **Phase 1 (PREPARE):** Check inventory availability and reserve stock
- **Phase 2 (COMMIT):** Create the order and deduct stock from inventory
- **Rollback:** On failure, release reservations and mark order as failed

2.3. Failover and High Availability

The methodology includes both **manual and automated failover simulation**:

- Health probing of database nodes
- Replica promotion for PostgreSQL

- Replica set reconfiguration for MongoDB
- Graceful degradation with partial database availability

3. Tech Stack

3.1. Backend

- **Framework:** Flask 2.3.3 (Python)
- **Database Drivers:**
 - psycopg2-binary==2.9.7 (PostgreSQL adapter)
 - pg8000==1.31.2 (Pure Python PostgreSQL adapter — fallback)
 - pymongo==4.5.0 (MongoDB adapter)
- **Authentication:** JWT using itsdangerous (URLSafeTimedSerializer)
- **CORS:** Flask-CORS 4.0.0
- **Environment:** python-dotenv 1.0.0

3.2. Frontend

- **Technology:** Static HTML/CSS/JavaScript (no framework)
- **Pages:** index.html, login.html, register.html, cart.html, product.html, profile.html, admin.html
- **Styling:** Custom CSS (styles.css)

3.3. Infrastructure

- **Containerization:** Docker + Docker Compose
- **Database Images:**
 - PostgreSQL: postgres:15-alpine
 - MongoDB: mongo:7.0

4. How Fragmentation is Used

The project implements **Functional Fragmentation** by domain, where different services manage separate data domains across independent database instances:

Service	Database	Responsibility
User Service	PostgreSQL (ports 15432/15433)	User authentication, profiles, audit logs
Order Service	PostgreSQL (ports 15434/15435)	Order management, order items, status history
Inventory Service	PostgreSQL (ports 15436/15437)	Stock management, inventory transactions
Product Catalog Service	MongoDB (ports 37017/37018)	Product details, categories, reviews

4.1. Key Fragmentation Features

- Each service has its own dedicated PostgreSQL database (`user_service_db`, `order_service_db`, `inventory_service_db`)
- Product data is stored in MongoDB as a document store for schema flexibility
- Cross-service communication happens through the Flask API gateway (`app.py`)
- The **Two-Phase Commit (2PC)** simulation coordinates transactions across inventory and order services

4.2. 2PC Flow Implementation

1. **Phase 1 (PREPARE):** Check inventory and reserve stock using `check_and_reserve_stock()` procedure

2. **Phase 2 (COMMIT):** Create the order and deduct stock using `create_order()` and `deduct_stock()` procedures
3. **Rollback:** On failure, rollback reservations and mark order as `FAILED`

5. How Replication is Used

5.1. PostgreSQL Replication (Physical Streaming Replication)

- **Type:** Asynchronous hot standby streaming replication
- **Configuration:** Set via `wal_level=replica`, `max_wal_senders=10`, `max_replication_slots=10`
- **Method:** Uses `pg_basebackup` for initial replica synchronization

Primary/Replica Pairs:

- User DB: `user-db-primary` (15432) → `user-db-replica` (15433)
- Order DB: `order-db-primary` (15434) → `order-db-replica` (15435)
- Inventory DB: `inventory-db-primary` (15436) → `inventory-db-replica` (15437)

Replication Health Monitoring:

- `probe_postgres_node()` function checks `pg_is_in_recovery()` and WAL LSN positions
- `wait_for_pg_replica_catchup()` ensures replica catches up before promotion
- `pg_promote()` used to promote replica to primary

5.2. MongoDB Replication (Replica Set)

- **Type:** MongoDB replica set with oplog-based asynchronous replication
- **Configuration:** One primary (37017) and one secondary (37018)
- **Failover:** Uses `replSetStepUp` and `replSetStepDown` commands
- **Standalone Mode Fallback:** If not configured as replica set, uses active endpoint switching with manual data sync via `sync_mongo_standalone_data()`

6. Explanation of Fragmentation and Replication

6.1. Fragmentation

Fragmentation in distributed databases refers to the division of a database into smaller parts (fragments) that are stored across different database instances. In Distribuy, **functional fragmentation** is employed:

- The database is partitioned by **business domain** rather than by rows or columns
- Each service owns its data completely, enabling **independent scaling** and **fault isolation**
- Data that is frequently accessed together stays together (cohesion)
- Cross-service queries are minimized, reducing network overhead

Benefits of this approach:

- Improved **query performance** (each DB handles fewer tables)
- **Independent backup** and maintenance per service
- **Fault tolerance** — one service's database failure doesn't bring down others

6.2. Replication

Replication is the process of maintaining multiple copies of the same data across different database instances. Distribuy implements replication at two levels:

6.2.1. PostgreSQL Streaming Replication

Physical replication where the **Write-Ahead Log (WAL)** is streamed from primary to replica:

- **Asynchronous** — replica may lag slightly behind primary

- **Hot Standby** — replica can serve read-only queries
- **Failover** — replica can be promoted to primary if needed

6.2.2. MongoDB Replica Set

Logical replication using the **oplog** (operation log):

- **Automatic failover** — secondary is elected primary on failure
- **Read preference** — clients can read from secondary nodes
- **Data redundancy** — multiple copies ensure durability

Benefits of replication:

- **High availability** — system continues if primary fails
- **Read scaling** — read-only queries can be distributed
- **Disaster recovery** — data is preserved on multiple nodes

7. The Databases Used

7.1. PostgreSQL 15-Alpine (3 Instances)

User Service Database: user_service_db

- Tables: users, user_audit_log

Order Service Database: order_service_db

- Tables: orders, order_items, order_status_history, transactional_log

Inventory Service Database: inventory_service_db

- Tables: inventory, inventory_transactions

7.2. MongoDB 7.0 (1 Replica Set)

Database: product_catalog_db

- Collections: products, categories, reviews

8. Schema, Procedures, and Triggers

8.1. User Service Schema

```
-- Users table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    phone VARCHAR(20),
    address TEXT,
    city VARCHAR(100),
    state VARCHAR(100),
    zip_code VARCHAR(10),
    country VARCHAR(100),
    is_admin BOOLEAN DEFAULT FALSE,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_login TIMESTAMP
);

-- Audit log table
CREATE TABLE user_audit_log (
    id SERIAL PRIMARY KEY,
    user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
    action VARCHAR(50) NOT NULL,
    old_data JSONB,
```

```

        new_data JSONB,
        changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    );

```

Triggers:

- trigger_update_users_updated_at — Updates updated_at timestamp on user changes
- trigger_log_user_changes — Logs all INSERT/UPDATE/DELETE operations to user_audit_log (skips no-op updates)

Stored Procedures:

- register_user(name, email, password, phone) — User registration with validation
- authenticate_user(email, password) — User login with password check
- get_user_profile(user_id) — Retrieve user profile
- update_user_profile(...) — Update user profile fields
- get_all_users(limit, offset) — Admin: list all users
- toggle_user_status(user_id) — Admin: toggle active/inactive status

8.2. Order Service Schema

-- Orders table

```

CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    total_price DECIMAL(12,2) NOT NULL,
    status VARCHAR(50) DEFAULT 'PENDING' CHECK (status IN
('PENDING', 'CONFIRMED', 'PROCESSING', 'SHIPPED', 'DELIVERED', 'CANCELLED', 'FAILED')),
    payment_method VARCHAR(50),
    shipping_address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completed_at TIMESTAMP
);

```

-- Order items table

```

CREATE TABLE order_items (
    id SERIAL PRIMARY KEY,
    order_id INTEGER NOT NULL REFERENCES orders(id) ON DELETE CASCADE,
    product_id INTEGER NOT NULL,
    product_name VARCHAR(255),
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    unit_price DECIMAL(12,2) NOT NULL,
    subtotal DECIMAL(12,2) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Triggers:

- trigger_update_orders_updated_at — Updates timestamp on order changes
- trigger_log_order_status — Logs status changes to order_status_history

Stored Procedures:

- create_order(user_id, items_json, shipping_address, payment_method) — Creates order with 2PC simulation
- get_order_details(order_id) — Get complete order information
- get_user_orders(user_id, limit, offset) — List orders for a user
- update_order_status(order_id, new_status, reason) — Update order status
- get_order_history(order_id) — Retrieve status change history

- `simulate_2pc_failure(order_id)` — Debugging: simulate transaction failure

8.3. Inventory Service Schema

```
CREATE TABLE inventory (
  id SERIAL PRIMARY KEY,
  product_id INTEGER NOT NULL UNIQUE,
  product_name VARCHAR(255),
  stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0),
  reserved_quantity INTEGER NOT NULL DEFAULT 0 CHECK (reserved_quantity >= 0),
  warehouse_location VARCHAR(255),
  reorder_level INTEGER DEFAULT 10,
  last_restock_date TIMESTAMP,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE inventory_transactions (
  id SERIAL PRIMARY KEY,
  product_id INTEGER NOT NULL,
  transaction_type VARCHAR(50),
  quantity_changed INTEGER NOT NULL,
  old_quantity INTEGER,
  new_quantity INTEGER,
  reference_id VARCHAR(255),
  notes TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  created_by VARCHAR(255)
);
```

Triggers:

- `trigger_update_inventory_updated_at` — Updates timestamp
- `trigger_prevent_negative_stock` — Prevents negative stock/reserved quantities
- `trigger_check_reorder_level` — Creates alert transaction when stock falls below reorder level

Stored Procedures:

- `check_and_reserve_stock(product_id, quantity)` — Uses `SELECT FOR UPDATE` for row-level locking
- `deduct_stock(product_id, quantity, order_id)` — Confirm sale and deduct stock
- `get_inventory_status(product_id)` — Get current inventory levels
- `restock_inventory(product_id, quantity, notes)` — Add stock to inventory
- `unreserve_stock(product_id, quantity)` — Release reserved stock (order cancelled)
- `get_low_stock_items()` — List items at or below reorder level

8.4. MongoDB Schema

```
// Products collection
{
  _id: 1,
  name: "Wireless Headphones",
  description: "...",
  price: 129.99,
  category: "Electronics",
  stock_reference_id: 1, // Links to inventory.product_id
  brand: "TechBrand",
  ratings: 4.5,
  reviews_count: 245,
```

```

    images: ["/images/headphones1.jpg", "..."],
    specifications: { battery_life: "40 hours", ... },
    is_active: true,
    createdAt: new Date(),
    updatedAt: new Date()
  }
}

```

Indexes:

- Products: {name:1}, {category:1}, {price:1}, {createdAt:-1}, {stock_reference_id:1}, {'\$**':'text'} (full-text search)
- Categories: {name:1} (unique)
- Reviews: {product_id:1}, {user_id:1}, {createdAt:-1}

9. Advantages of This System

1. **Educational Value** — Excellent demonstration of distributed database concepts (fragmentation, replication, 2PC)
2. **Modular Architecture** — Clear separation of concerns by service domain
3. **Real Database Features** — Uses actual PostgreSQL streaming replication and MongoDB replica sets
4. **Failover Support** — Includes both manual and automated failover simulation
5. **Comprehensive Monitoring** — Admin dashboard shows node health, replication status, WAL LSN positions
6. **Audit Trails** — Audit logs for user changes and order status history
7. **Row-Level Locking** — Uses `SELECT FOR UPDATE` for concurrency control in inventory
8. **Data Integrity** — CHECK constraints, foreign keys, and triggers prevent invalid data
9. **Graceful Degradation** — System continues operating with partial database availability
10. **Connection Resilience** — Automatic reconnection and fallback to replica endpoints

10. Limitations

1. **Password Security** — Password hashing should be enabled before production use; currently stores plain text passwords
2. **No Real Auto-Failback for PostgreSQL** — After replica promotion, system intentionally avoids auto-switching back to original primary (to prevent stale data)
3. **Simulation vs. Production:**
 - 2PC is simulated, not true distributed transactions
 - Failover is container-level, not true cluster management
 - MongoDB standalone mode has no oplog replication (manual sync required)
4. **Missing Production Features** — Needed additions include:
 - External secret management
 - Real replica promotion automation
 - Dedicated API gateway / reverse proxy hardening
 - Monitoring and alerting systems
 - Integration tests for 2PC and failover flows
5. **Standalone MongoDB Mode** — When not running replica sets, requires manual `sync_mongo_standalone_data()` to sync data between nodes

6. **No Sharding** — Only replication is demonstrated; no horizontal partitioning (sharding)
7. **Frontend Simplicity** — Static HTML/JS without modern framework (limits scalability for complex UIs)

11. Conclusion

The Distribuy project successfully demonstrates the core concepts of distributed database systems including functional fragmentation, replication, and distributed transaction management. While designed primarily for educational purposes, the architecture follows sound distributed systems principles and provides a solid foundation for understanding how modern e-commerce platforms can leverage distributed databases for scalability and availability.

The modular design with separate database instances per service, combined with replication for fault tolerance, illustrates a practical approach to building resilient distributed applications. Future enhancements could include production-grade security, true distributed transactions, and sharding for horizontal scalability.